Healthcare Analysis using Big Data Analytics

Akash Samanta – 2025-MPCS-001
Harsh Wadhwani – 2025-MPCS-003

Department of Computer Science and Engineering
Atal Bihari Vajpayee - Indian Institute of Information Technology and Management
Gwalior

Table of Contents

1. Introduction
2. Dataset and Technologies
3. System Architecture
4. Dataset and Data Processing
5. Data Ingestion and Preprocessing
6. Readmission Labeling
7. Feature Engineering and Dataset Preparation
8. Model Training and Evaluation

Introduction

- Healthcare institutions generate large volumes of clinical and administrative data.
- This project implements a healthcare data analysis pipeline using Hadoop, Hive, and Apache Spark in a Docker-based environment.
- Analysing this data can help identify patient characteristics, hospital utilisation patterns, and clinically relevant trends.
- Big Data technologies provide tools for storing, processing, and analysing structured healthcare datasets.

Project Objectives

1. Set up a containerised Big Data processing environment using **Docker Compose**.
2. Store and manage healthcare data using **Hadoop Distributed File System (HDFS).**
3. Process and analyse healthcare records using **Apache Spark** and **Spark SQL**.
4. Use **Hive** to support **Spark SQL**-based data analysis and metadata management.
5. Investigate patient admission patterns, ICU utilisation, and hospital readmissions.
6. Explore machine learning techniques - **Spark MLlib** - for predicting 30-day hospital readmission.

Dataset and Technologies

Healthcare Dataset

- MIMIC clinical dataset
- Patient demographics, Hospital admissions, Diagnoses and ICU records

Technology Stack

- Docker
- Hadoop HDFS, YARN
- Apache Hive, Spark, Spark SQL, Spark MLlib

Dataset Note

The Medical Information Mart for Intensive Care (MIMIC-IV) database is comprised of de-identified electronic health records for patients admitted to the Beth Israel Deaconess Medical Center. Access to MIMIC-IV is limited to credentialed users. Here, we have used an openly-available demo of MIMIC-IV containing a subset of 100 patients.
Source : https://doi.org/10.13026/07hj-2a80

Docker-Based System Architecture

Container Orchestration

Docker

Hadoop Services

- NameNode
- DataNode
- YARN ResourceManager
- YARN NodeManager

Analytics Services

- Hive Metastore
- HiveServer2
- PostgreSQL Metastore DB
- Spark Master, Worker

Shared Project Workspace

The Compose configuration mounts the project workspace into the NameNode, HiveServer2, and Spark Master containers. HDFS data is stored using Docker volumes.

Cluster Services and Interfaces

Component	Purpose	Port
HDFS NameNode	HDFS web interface	9870
HDFS RPC	HDFS client access	8020
YARN ResourceManager	Resource management UI	8088
Hive Metastore	Metadata service	9083
HiveServer2	SQL client access	10000
Spark Master	Spark web interface	8080
Spark Master	Cluster communication	7077

Docker Compose Configuration

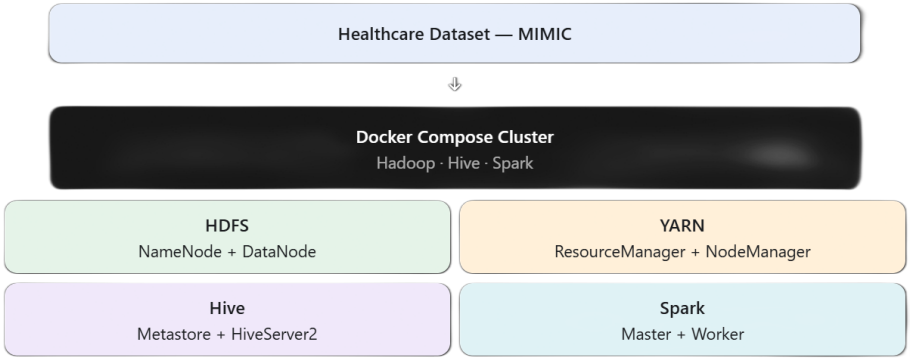


Figure: Hadoop Architecture

Docker Deployment

```
$ docker compose up -d
[+] up 9/9
 ✓ Container hive-metastore              Started
 ✓ Container namenode                    Started
 ✓ Container hiveserver2                 Started
 ✓ Container spark-master                Started
 ✓ Container hive-metastore-postgresql   Started
 ✓ Container datanode                    Started
 ✓ Container spark-worker                Started
 ✓ Container resourcemanager             Started
 ✓ Container nodemanager                 Started
```

Figure: Container Running in Docker

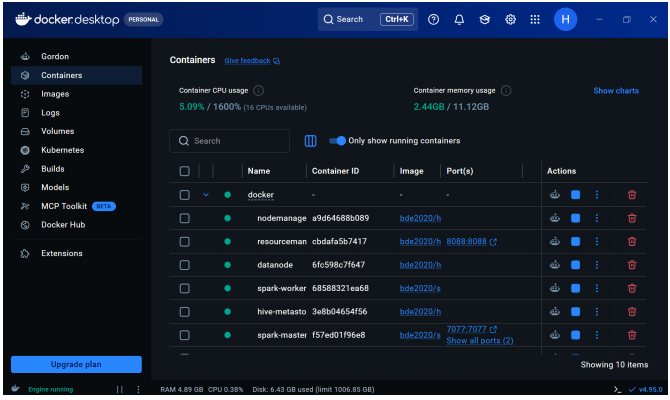


Figure: Docker Visualization

Docker Deployment



Figure: Processes in Docker

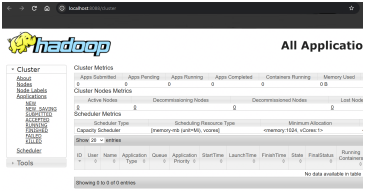


Figure: YARN

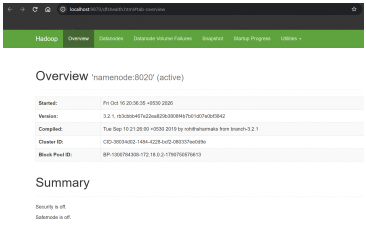


Figure: NAMENODE

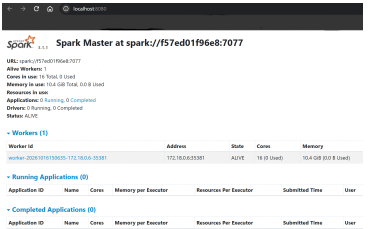


Figure: SPARK

Dataset Description

Dataset Overview

The project uses the Medical Information Mart for Intensive Care (MIMIC) dataset for healthcare data analysis.

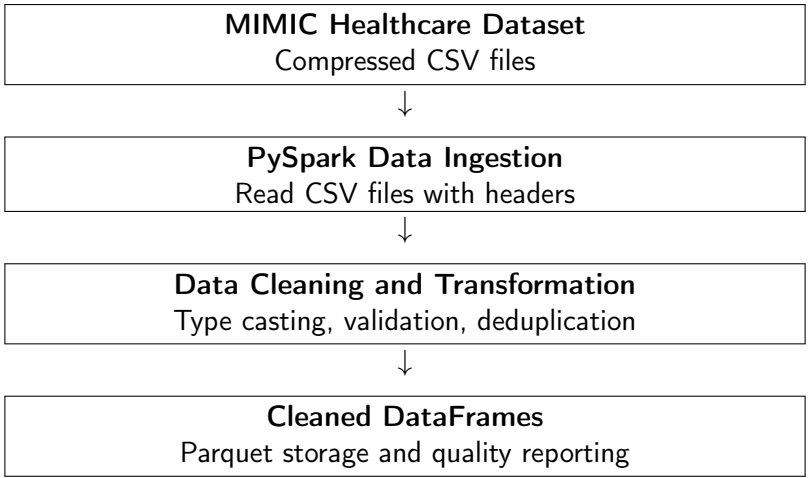
Hospital Data

- Patient demographics
- Hospital admissions
- Diagnoses
- Admission and discharge information

ICU Data

- ICU stays
- ICU patient information
- ICU length of stay

Data Workflow



Data Cleaning and Preprocessing

Dataset	Preprocessing Operations
Admissions	Cast identifiers and timestamps; retain records where discharge time is later than admission time; remove duplicate admission IDs; derive length of stay in days.
Patients	Cast patient identifiers and age/year fields; remove duplicate patient IDs.
Diagnoses	Cast patient, admission, sequence, and ICD version fields; remove duplicates using admission ID and sequence number.
ICD Descriptions	Cast ICD version and remove duplicate ICD code/version combinations.
ICU Stays	Cast identifiers and timestamps; retain valid ICU time intervals; remove duplicate stay IDs.
Chart Events	Retain selected vital-sign item IDs, non-null numeric measurements, and records with non-null ICU stay IDs.

Cleaning rules are applied using the PySpark DataFrame API.

Data Cleaning Results

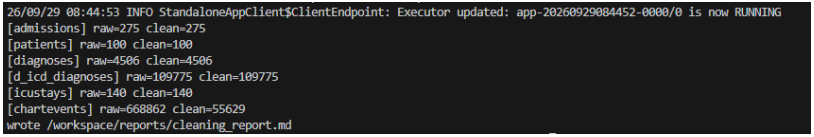


Figure: Cleaning Results

Table	Raw Rows	Clean Rows	Dropped
Admissions	275	275	0
Patients	100	100	0
Diagnoses	4,506	4,506	0
ICD Descriptions	109,775	109,775	0
ICU Stays	140	140	0
Chart Events	668,862	55,629	613,233

Key Observation

Five tables retained all their rows after the specified cleaning operations. The chart events table underwent substantial filtering based on the selected vital-sign item IDs and data validity conditions.

Cleaned Data Storage

Storage Format

Cleaned Spark DataFrames are written in Apache Parquet format for downstream analytical processing.

Tables written by the pipeline

- `admissions`
- `patients`
- `diagnoses`
- `d_icd_diagnoses`
- `icustays`
- `chartevents`

Implementation

Each cleaned DataFrame is saved to `CLEAN/<table_name>` using `write.mode("overwrite").parquet(...)`.

```
0: jdbc:hive2://localhost:10000> SHOW TABLES IN mimic;
+---------------------+
|      tab_name       |
+---------------------+
| admissions_labeled  |
| d_icd_diagnoses     |
| diagnoses_icd       |
| icustays_clean      |
| patients_clean      |
+---------------------+
5 rows selected (1.356 seconds)
0: jdbc:hive2://localhost:10000> SELECT COUNT(*) AS n_admissions FROM mimic.admissions_labeled;
WARNING: Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engin
e (i.e. spark, tez) or using Hive 1.X releases.
+--------------+
| n_admissions |
+--------------+
| 275          |
+--------------+
1 row selected (6.266 seconds)
0: jdbc:hive2://localhost:10000>
0: jdbc:hive2://localhost:10000> Closing: 0: jdbc:hive2://localhost:10000
PS C:\Users\Akash Samanta\Documents\Projects\Healthcare-Analysis-Big-Data\docker>
```

Figure: Created Tables

Readmission Labeling

- The cleaned admissions dataset is used to construct a 30-day readmission target.
- A PySpark window function identifies each patient's next admission in chronological order.
- The resulting `readmit_30d` label and related admission features are saved as a Parquet dataset.

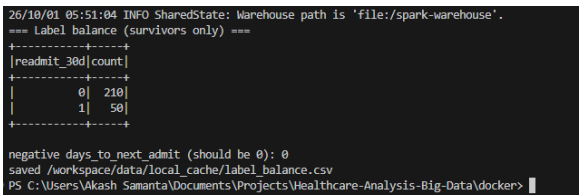


Figure: Label Balance

Readmission Labeling Algorithm

Label Construction Algorithm

1. For each patient, sort admissions chronologically.
2. For each admission, assign label $y \in \{0, 1\}$:
   - $y = 1$ if the patient survived (`hospital_expire_flag` $= 0$), the next admission begins within 30 days after discharge, and the next admission is not `ELECTIVE`.
   - $y = 0$ otherwise.
3. Assign $y = 0$ to the last admission of each patient (right-censoring; a study limitation).

Label Distribution

Negative: 210 Positive: 50 Total: 260

Positive rate: $\frac{50}{260} \times 100 \approx 19.2\%$

Spark SQL: ICU Utilization and Outcomes

The script performs grouped analysis to examine ICU stays across different patient and admission categories by Number of stays, average ICU length of stay, and hospital mortality rate.

- **By first care unit**
- **By admission type**
- **By age band**
- **Length-of-stay distribution**

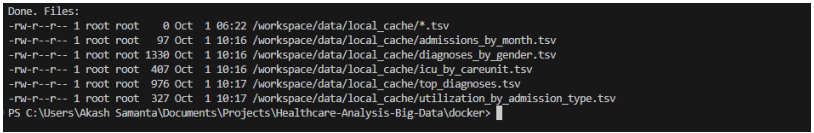
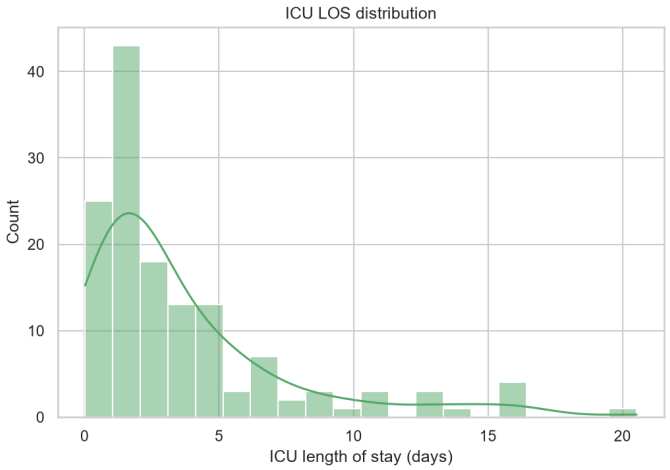
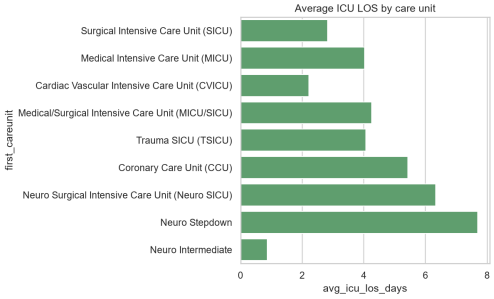


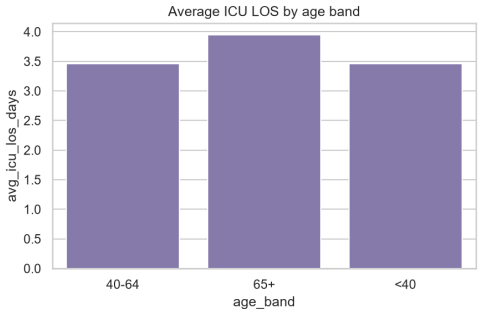
Figure: Output generated after ICU

ICU Length of Stay Distribution



Average ICU Length of Stay

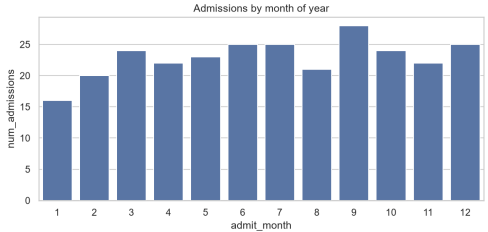


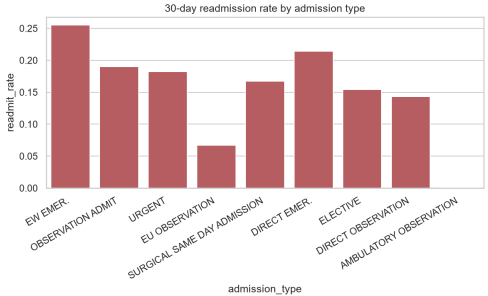


Feature Engineering for Readmission Prediction

- The labeled admissions dataset is combined with patient demographics, diagnosis counts, and ICU-derived features.
- Only admissions with `hospital_expire_flag = 0` are included in the modeling cohort.
- The following features are constructed:
  - **Demographic:** age and gender.
  - **Admission:** admission type, insurance, and length of stay.
  - **Historical:** number of prior admissions.
  - **Clinical:** number of diagnoses and ICU stay information.
  - **Vital signs:** average heart rate, respiratory rate, and SpO_2 during the first 24 hours.
- Race is excluded from the feature set because the code identifies rare categories and fairness considerations as concerns.

Readmissions by Month & Admission type





Patient-Level Stratified Train-Test Split

- The dataset is split into training and test subsets at the patient level.
- Stratification uses each patient's `ever_pos` indicator, which identifies whether that patient has any positive readmission label.
- An 80% sampling fraction is used for each patient-level stratum, with random seed 42.

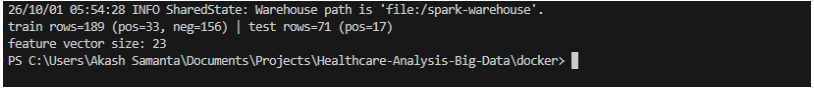
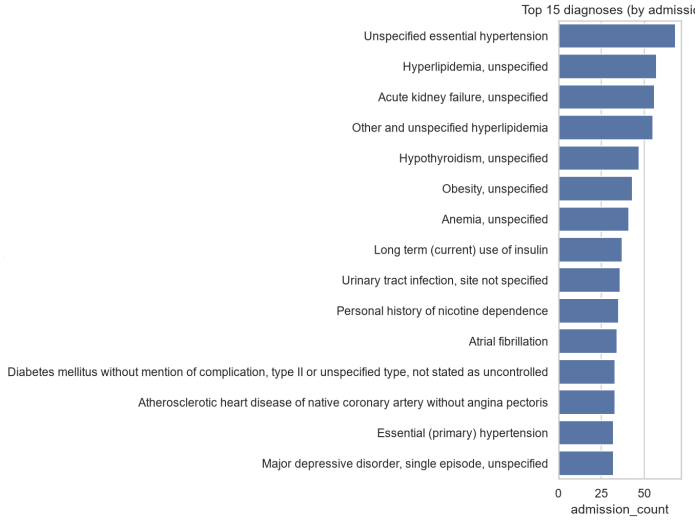


Figure: Train-test split

Leakage prevention

The feature preprocessing pipeline is fitted on the training data only, then applied to both datasets.

Top 15 Diagnoses



Machine Learning Models

- The prepared training and test datasets are loaded from the feature-engineering stage.
- Three binary classification algorithms are trained to predict 30-day unplanned hospital readmission:
  - **Logistic Regression (LR):** linear classification baseline.
  - **Random Forest (RF):** ensemble of decision trees.
  - **Gradient-Boosted Trees (GBT):** sequentially boosted decision trees.
- All three models use the prepared feature vectors and training class weights & evaluated on the held-out test set.

Model	Configuration
Logistic Regression	Maximum 50 iterations; regularization = 0.1
Random Forest	Seed = 42; 3-fold cross-validation when training positives ≥ 9
Random Forest tuning	Trees: 50, 100; maximum depth: 3, 5
Random Forest fallback	100 trees; maximum depth = 4
Gradient-Boosted Trees	30 iterations; maximum depth = 3; seed = 42

- Random Forest hyperparameters are selected using cross-validation with area under the precision-recall curve as the evaluator.

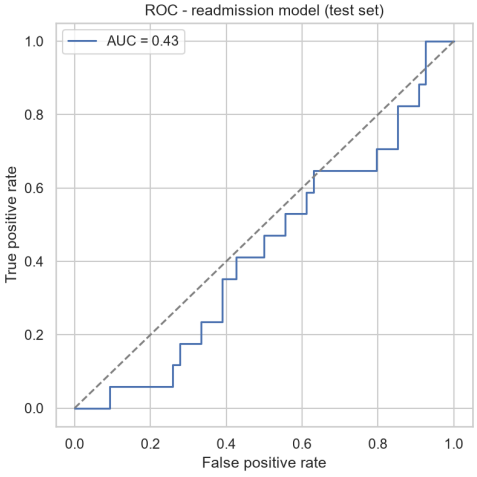
Model Evaluation and Results

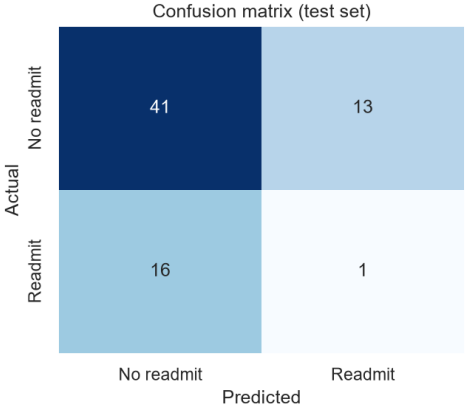
Model	Test AUC	Train AUC	Prec.	Recall	F1
Logistic Regression	0.466	0.793	0.167	0.176	0.171
Random Forest	0.428	0.919	0.071	0.059	0.065
Gradient-Boosted Trees	0.449	0.991	0.214	0.176	0.194

Honest interpretation

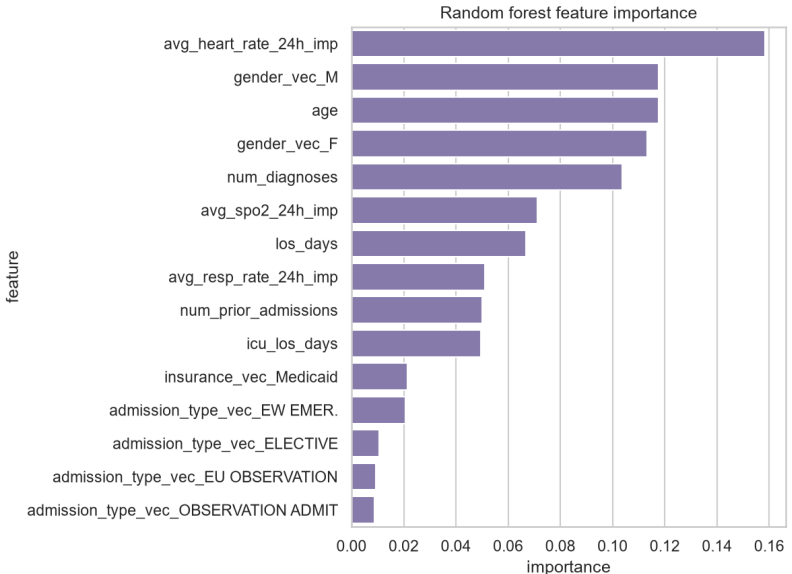
Test AUCs are at or below 0.5 while train AUCs are high: classic overfitting with only about 33 positive training cases. This reflects the demo dataset size, not a pipeline error; the same code on full MIMIC-IV would have far more positives.

Results


ROC - readmission model (test set)
AUC = 0.43
True positive rate
False positive rate
0.0
0.2
0.4
0.6
0.8
1.0


Confusion matrix (test set)
Actual
No readmit
Readmit
41
13
16
1
No readmit
Readmit
Predicted

Feature Importance



Contributions at a Glance

Stage	Harsh Wadhwani	Akash Samanta
Docker cluster, HDFS, YARN	Compose file, 9 containers, HDFS load	–
Ingestion and cleaning (Spark)	Schemas, cleaning, Parquet, cleaning report	–
Readmission label	–	Definition and validation
Hive disease / utilization	DDL and 5 HiveQL queries	–
ICU analysis (Spark SQL)	First-24h vitals, outcome analysis, `icu_by_hadm`	–
Feature engineering (MLlib)	–	Features, pipeline, patient-level split, class weights
Model training and evaluation	–	LR, RF, GBT, CV, metrics, feature importance
Visualization	Plotting script, 9 figures	

Conclusion and Future Work

- Delivered a reproducible pipeline: ingestion, cleaning, labeling, Hive analytics, ICU analysis, feature engineering, three MLlib models, visualization.
- Future work: run on full MIMIC-IV (60,000+ patients), add clinical notes with NLP, streaming ingestion, fairness audit.